

Competitive Programming

Section 34 Editorial: Probability and Expected Value

Competitive Programming Course

2026-07-10

Table of contents

1	Editorial overview	3
2	A. Expected Prize	4
2.1	Restatement and constraints	4
2.2	Derivation	4
2.3	Worked trace	4
2.4	Algorithm and proof	4
2.5	Complexity, language notes, and tests	5
3	B. First Success	6
3.1	Restatement and constraints	6
3.2	Derivation and trace	6
3.3	Algorithm and proof	6
3.4	Complexity, language notes, and tests	6
4	C. Coupon Collector	7
4.1	Restatement and constraints	7
4.2	Derivation and complete trace	7
4.3	Algorithm	7
4.4	Correctness proof	8
4.5	Complexity, language notes, and tests	8
5	Local workflow	9
5.1	A. Expected Prize: C++	9
5.2	A. Expected Prize: Python	10
5.3	B. First Success: C++	10
5.4	B. First Success: Python	11
5.5	C. Coupon Collector: C++	11
5.6	C. Coupon Collector: Python	12
6	What to carry forward	13

1 Editorial overview

Problem	Random-variable model	Main tool	Cost
A. Expected Prize	one reward variable per event	linearity	$O(n \log \text{MOD})$
B. First Success	expected remaining attempts	first-step equation	$O(t \log \text{MOD})$
C. Coupon Collector	wait for each new type	staged linearity	$O(\max n + q)$

All answers are rational numbers represented modulo the prime $\text{MOD} = 1,000,000,007$. The local denominators are strictly below MOD, so their Fermat inverses exist.

2 A. Expected Prize

2.1 Restatement and constraints

There are up to 200,000 bonus events. Event i awards value v_i with probability p_i/q_i and awards zero otherwise. The statement calls events independent, but the solution only needs their individual probabilities and q_i not divisible by MOD.

2.2 Derivation

Define X_i to be the points from event i . It is v_i with probability p_i/q_i and zero otherwise, hence $E[X_i] = v_i * p_i / q_i$. The total score is the sum of the X_i , so linearity gives

$$E[\text{total}] = \sum v_i * p_i * \text{inverse}(q_i) \text{ modulo MOD}.$$

Linearity does not require independence. This is why we sum event contributions instead of enumerating all 2^n possible subsets.

2.3 Worked trace

For sample events (1,2,10), (1,4,8), and (3,4,12):

$$\begin{aligned} E[X_1] &= 10 * 1/2 = 5 \\ E[X_2] &= 8 * 1/4 = 2 \\ E[X_3] &= 12 * 3/4 = 9 \\ E[\text{total}] &= 16 \end{aligned}$$

If the third event were correlated with the first two but still had probability 3/4, the same expected total would follow. Independence would matter for a product of probabilities, not this sum.

2.4 Algorithm and proof

```
answer = 0
for each p,q,v:
    answer += (v modulo MOD) * (p modulo MOD) * q^(MOD-2)
    answer %= MOD
print answer
```

For each event this expression is its exact expected contribution in the finite field. By linearity, summing all contributions equals the expected total. The inverse is valid because $1 \leq q < \text{MOD}$. Therefore the algorithm is correct. QED.

2.5 Complexity, language notes, and tests

Time is $O(n \log \text{MOD})$, memory $O(1)$ besides input/output buffering. C++ products must use long long. Python should use the three-argument pow function; ordinary division and floating point both lose the exact residue.

Test one certain bonus, a zero-probability bonus, a zero value, and a value larger than MOD. Frequent wrong answers multiply all event probabilities or incorrectly think dependent events invalidate linearity.

3 B. First Success

3.1 Restatement and constraints

For each of up to 200,000 queries, independent attempts succeed with probability p/q , where $1 \leq p \leq q < \text{MOD}$. Find the expected attempts through the first success. The lower bound $p \geq 1$ is essential: probability zero would mean an infinite waiting time, not answer zero.

3.2 Derivation and trace

Let E be the expected attempts before an attempt. One attempt is made now. If it fails, probability $(q-p)/q$, the state is exactly the same as before:

$$\begin{aligned} E &= 1 + ((q-p)/q) E \\ (p/q) E &= 1 \\ E &= q/p \end{aligned}$$

For $p=2, q=5$:

$$\begin{aligned} E &= 1 + (3/5)E \\ (2/5)E &= 1 \\ E &= 5/2 \end{aligned}$$

The modular output is 5 times $\text{inverse}(2)$, namely 500000006.

3.3 Algorithm and proof

For every query, print $q * p^{-1} \pmod{\text{MOD}}$. The first-step equation conditions on all possible results of the first attempt and uniquely solves to q/p . Since p is nonzero modulo MOD , its inverse represents that rational value exactly. Thus the algorithm is correct. QED.

3.4 Complexity, language notes, and tests

Time is $O(t \log \text{MOD})$, memory $O(1)$ besides output. In C++, the product of two reduced residues fits in signed 64-bit. In Python, $\text{pow}(p, \text{MOD}-2, \text{MOD})$ is the right inverse operation.

Test $p=q$, $p=1$, $p=1$ and $q=2$, and large p and q close to MOD . Wrong submissions usually output p/q , omit the leading one in the recurrence, or treat $p=0$ as a valid zero expectation.

4 C. Coupon Collector

4.1 Restatement and constraints

For each of up to 200,000 queries, n is at most one million. Each draw returns one of n coupon types uniformly and independently. We need the expected draws until every type has appeared. Repeating a computation for each query is too slow, but all queries share harmonic prefixes.

4.2 Derivation and complete trace

At stage i , exactly i types are already known. A draw finds a new type with probability $(n-i)/n$, so its geometric waiting-time expectation is $n/(n-i)$. Adding all stages gives

$$\begin{aligned} E &= n/n + n/(n-1) + \dots + n/1 \\ &= n * (1 + 1/2 + \dots + 1/n). \end{aligned}$$

For $n=3$:

stage	success probability	expected draws
0 -> 1	3/3	1
1 -> 2	2/3	3/2
2 -> 3	1/3	3
total		11/2

The required residue is 3 times $(1 + \text{inverse}(2) + \text{inverse}(3))$, equal to 500000009. The stage random variables are added with linearity.

4.3 Algorithm

Read all queries and let limit be their maximum. Precompute inverses and harmonic prefixes in linear time:

```
inv[1] = 1
H[0] = 0
for i from 1 to limit:
    if i >= 2:
        inv[i] = MOD - (MOD/i) * inv[MOD%i] modulo MOD
        H[i] = H[i-1] + inv[i] modulo MOD
for n in queries:
    print n * H[n] modulo MOD
```

The inverse recurrence is valid for $1 \leq i < \text{MOD}$, precisely the local bound. It is preferable to one modular exponentiation per i .

4.4 Correctness proof

During stage i , the probability of moving to stage $i+1$ is $(n-i)/n$, so the First Success result gives an expected stage length $n/(n-i)$. Completion is the sum of exactly these n stages. Linearity gives n times the harmonic sum. The precomputation stores every required modular reciprocal and prefix sum, so each answer is that exact rational expectation modulo MOD. QED.

4.5 Complexity, language notes, and tests

Preprocessing costs $O(\max n)$, each query $O(1)$, and memory $O(\max n)$. C++ should cast the inverse-recurrence product to long long. Python lists are practical at one million entries. A pow-per-inverse loop is correct but $O(\max n \log \text{MOD})$ and does unnecessary work.

Test $n=1$, $n=2$, $n=3$, consecutive queries, and the maximum query. Common wrong answers use n/i instead of $n/(n-i)$, omit the first stage, or recompute a harmonic sum independently per query.

5 Local workflow

Run:

```
CP_TARGET=solution python3 sections/34_probability_expected_value/check.py
```

For small verification, use exact rational arithmetic. Enumerate a few reward outcomes, solve the first-step equation symbolically, and compare coupon stage sums using Python Fraction before reducing numerator times inverse(denominator) modulo MOD. # Full reference implementations

5.1 A. Expected Prize: C++

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long result = 1;
    while (b > 0) {
        if (b & 1LL) result = result * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    long long ans = 0;
    for (int i = 0; i < n; i++) {
        long long p, q, v;
        cin >> p >> q >> v;
        ans = (ans + (v % MOD) * (p % MOD) % MOD * mod_pow(q, MOD - 2)) % MOD;
    }
    cout << ans << '\n';
    return 0;
}
```

5.2 A. Expected Prize: Python

```
import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    idx = 1
    ans = 0
    for _ in range(n):
        p, q, v = data[idx], data[idx + 1], data[idx + 2]
        idx += 3
        ans = (ans + v % MOD * p % MOD * pow(q, MOD - 2, MOD)) % MOD
    print(ans)

if __name__ == "__main__":
    main()
```

5.3 B. First Success: C++

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long result = 1;
    while (b > 0) {
        if (b & 1LL) result = result * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    if (!(cin >> t)) return 0;
```

```

while (t--) {
    long long p, q;
    cin >> p >> q;
    cout << q % MOD * mod_pow(p, MOD - 2) % MOD << '\n';
}
return 0;
}

```

5.4 B. First Success: Python

```

import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out = []
    for _ in range(t):
        p, q = data[idx], data[idx + 1]
        idx += 2
        out.append(str(q % MOD * pow(p, MOD - 2, MOD) % MOD))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

5.5 C. Coupon Collector: C++

```

#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long result = 1;
    while (b > 0) {
        if (b & 1LL) result = result * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
}

```

```

    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    vector<int> values(q);
    int limit = 0;
    for (int &n : values) {
        cin >> n;
        limit = max(limit, n);
    }
    vector<long long> harmonic(limit + 1, 0);
    for (int i = 1; i <= limit; i++) {
        harmonic[i] = (harmonic[i - 1] + mod_pow(i, MOD - 2)) % MOD;
    }
    for (int n : values) cout << n * harmonic[n] % MOD << '\n';
    return 0;
}

```

5.6 C. Coupon Collector: Python

```

import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    values = data[1 : 1 + q]
    limit = max(values)
    inv = [0] * (limit + 1)
    harmonic = [0] * (limit + 1)
    for i in range(1, limit + 1):
        inv[i] = pow(i, MOD - 2, MOD)
        harmonic[i] = (harmonic[i - 1] + inv[i]) % MOD
    print("\n".join(str(n * harmonic[n] % MOD) for n in values))

if __name__ == "__main__":
    main()

```

6 What to carry forward

Define the random variable or expected remaining state first. Linearity avoids distribution enumeration, and a first-step equation exposes every self-loop.